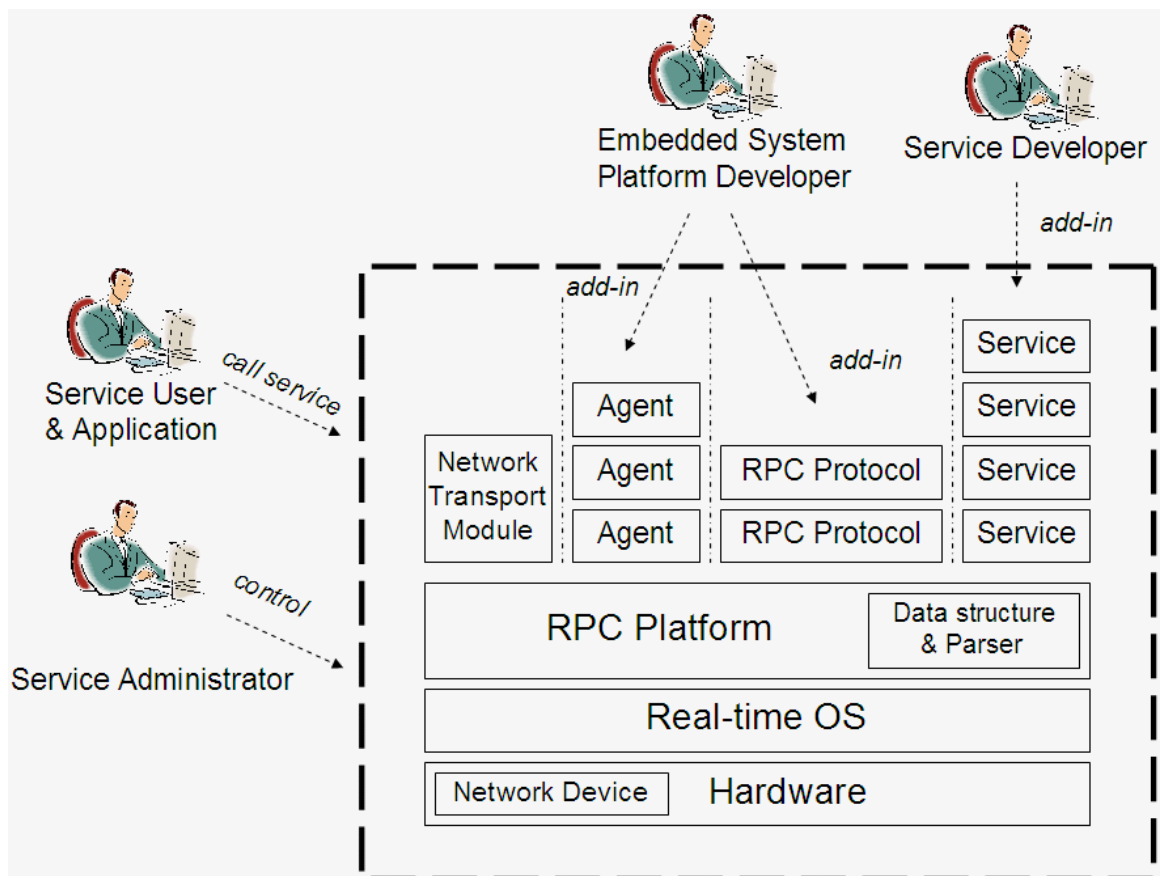


บทที่ 4

ลักษณะการใช้งานเว็บเซอร์วิสบนอุปกรณ์ฝังตัว

ลักษณะการใช้งานเว็บเซอร์วิสบนระบบฝังตัวนั้นสามารถแบ่งได้ออกเป็น 4 ลักษณะ คือ

1. นักพัฒนา
2. นักพัฒนาบริการบนอุปกรณ์ฝังตัว
3. ผู้ที่เรียกใช้บริการของระบบ
4. ผู้ดูแลระบบการให้บริการของระบบ



รูปภาพ 4-1 บทบาทของผู้ที่เกี่ยวข้องกับระบบ

4.1 นักพัฒนา

เป็นผู้ที่นำโค้ดของระบบ ไปพัฒนาในส่วนต่างๆต่อไปใช้งานเฉพาะอย่างที่ต้องการ โดยสามารถเพิ่มส่วนต่างๆ ดังนี้

- **โปรโตคอลแฮนเดิลเลอร์ (Protocol Handler)**

เป็นส่วนที่รองรับการติดต่อในรูปแบบของโปรโตคอลในมาตรฐานต่างๆ โดยส่วนนี้ ได้ถูกออกแบบให้สามารถขยายและเพิ่มเติมโปรโตคอลแฮนเดิลเลอร์ได้โดยนักพัฒนา โดยมีรูปแบบดังนี้

Protocol Handler In Task()

```
{  
  RPCParam *param;  
  MessageQ *mq;  
  for(;;){  
    //wait for request  
    //handle protocol to parameterobject  
    //save in parameter object  
    //save session infomation in RPCObject  
    //post return to requested task  
  }  
}
```

รูปภาพ 4-2 โครงสร้างการพัฒนาโปรโตคอลแฮนเดิลเลอร์ในส่วนของการดึงข้อมูลมาสู่อาร์พีซีพารามิเตอร์

จากรูปโครงสร้างของโปรแกรมในส่วนของโปรโตคอลแฮนเดิลเลอร์นี้ เป็นส่วนที่ทำหน้าที่อ่านข้อมูลจากมาตรฐานโปรโตคอล แล้วสร้างเก็บอาร์พีซีพารามิเตอร์เพื่อเก็บข้อมูลที่อ่านได้ แล้วเก็บบันทึกไว้ในพารามิเตอร์ออบเจกต์นั้น แล้วทำการเก็บบันทึกค่าที่จำเป็นต้องใช้ในการติดต่อในเซสชันของโปรโตคอลนั้นเอาไว้ในอาร์พีซีออบเจกต์ เมื่อทำงานเสร็จแล้ว ก็ทำการส่งแมชเชอไปยังทาสก์ที่เรียกใช้เพื่อให้ทำงานในลำดับอื่นๆต่อไป

Protocol Handler Out Task()

```
{  
    RPCParam *result;  
    MessageQ *mq;  
    for(;;){  
        //wait for request  
        //encode and serial data to message protocol  
        //post return to requested task  
    }  
}
```

รูปภาพ 4-3 โครงสร้างของโปรโตคอลแฮนเดิลเลอร์สำหรับแปลงข้อมูลกลับไปเป็นเอกสารของโปรโตคอล

จากรูป เป็นโครงสร้างของโปรโตคอลแฮนเดิลเลอร์ที่ทำหน้าที่แปลงข้อมูลที่เป็นผลลัพธ์ของการเรียกใช้บริการ ให้อยู่ในรูปของเอกสารตามมาตรฐานของแต่ละโปรโตคอล แล้วส่งผลกลับไปยังทาสก์ที่ร้องขอต่อไป

เนื่องจากระบบอาร์พีซีนี้ มีรากฐานมาจากแนวความคิดของสถาปัตยกรรมการทำงานแบบเว็บเซอร์วิส และเป็นโครงงานที่พัฒนาต่อ จึงมีการดัดแปลงโซบโปรโตคอลแฮนเดิลเลอร์ มาเป็นโปรโตคอลแฮนเดิลเลอร์ที่เตรียมไว้ให้ในระบบแล้ว เพื่อสามารถนำไปใช้ทำการสื่อสารแบบเว็บเซอร์วิสได้

- เซิร์ฟเวอร์แดมอนทาสก์ (Server Daemon Task)

เป็นส่วนที่คอยรับการขอทำการเชื่อมต่อไปกับผู้ร้องขอบริการ ซึ่งต้องรูปแบบในการติดต่อในข้อมูลในระดับของเซสชัน (Session Layer) โดยรองรับโปรโตคอลที่ทำหน้าที่เป็นผู้รับส่งข้อมูล (Agent) ซึ่งได้แก่ โปรโตคอลเอชทีทีพี โปรโตคอลเอฟทีที หรือ โปรโตคอลเอสเอ็มทีที เป็นต้น ในการพัฒนาเซิร์ฟเวอร์แดมอนนั้น จะควรจะพัฒนาตามรูปแบบของโครงสร้างที่ได้ออกแบบไว้ตามหน้าที่การทำงานของเซิร์ฟเวอร์แดมอน ดังนี้

```
<serverd>_init();  
<serverd>_thread();  
<serverd>_close();  
do_read();
```

รูปภาพ 4-4 โครงสร้างฟังก์ชันภายใน ในการอิมพลีเมนต์เซิร์ฟเวอร์แดมอน

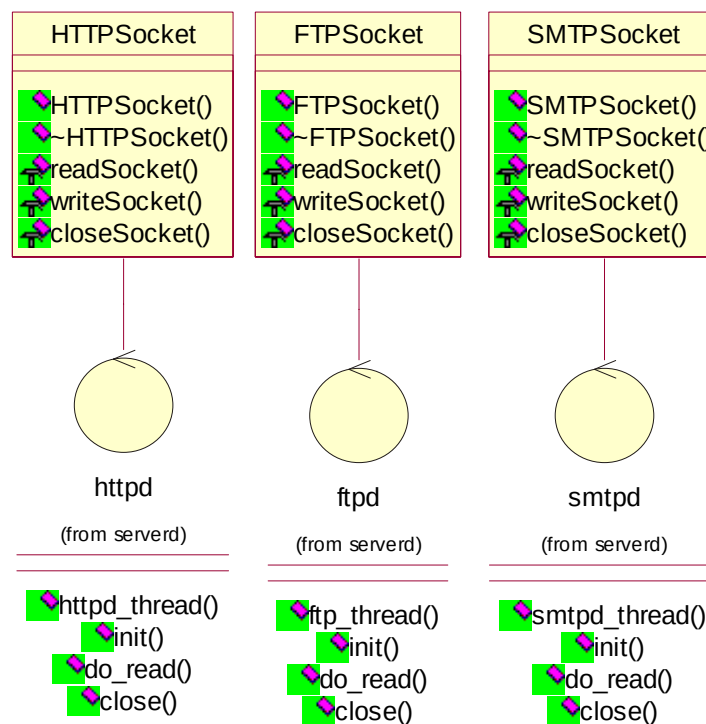
(หมายเหตุ : <serverd> คือ ชื่อของโปรโตคอลใดๆ ที่จะถูกอิมพลีเมนต์เข้ามาใช้ในระบบ)

<code><serverd>_init()</code>	เป็นฟังก์ชันที่เริ่มการทำงานของระบบ โดยจะเรียกฟังก์ชันของระบบที่ทำหน้าที่สร้างเทรด (thread) ของเซิร์ฟเวอร์แอดมอน เพื่อรอรับการร้องขอเชื่อมต่อใน โปรโตคอลที่เซิร์ฟเวอร์แอดมอนนั้นๆ รองรับ
<code><serverd>_thread()</code>	เป็นฟังก์ชันที่เป็นส่วนที่ประมวลผลของเทรดของเซิร์ฟเวอร์แอดมอน มีลักษณะคล้ายทาสก์ ที่วนรอบการทำงานตลอดเวลา
<code><serverd>_close()</code>	เป็นฟังก์ชันที่ปิดการเชื่อมต่อในเซสชันนั้นๆ
<code>do_read()</code>	เป็นฟังก์ชันในการอ่านค่าข้อมูลเข้ามาในเซิร์ฟเวอร์แอดมอนแล้วเก็บลงในโครงสร้างข้อมูลที่เตรียมไว้

ตาราง 4-1 อธิบายฟังก์ชันในการอิมพลีเมนต์เซิร์ฟเวอร์แอดมอน

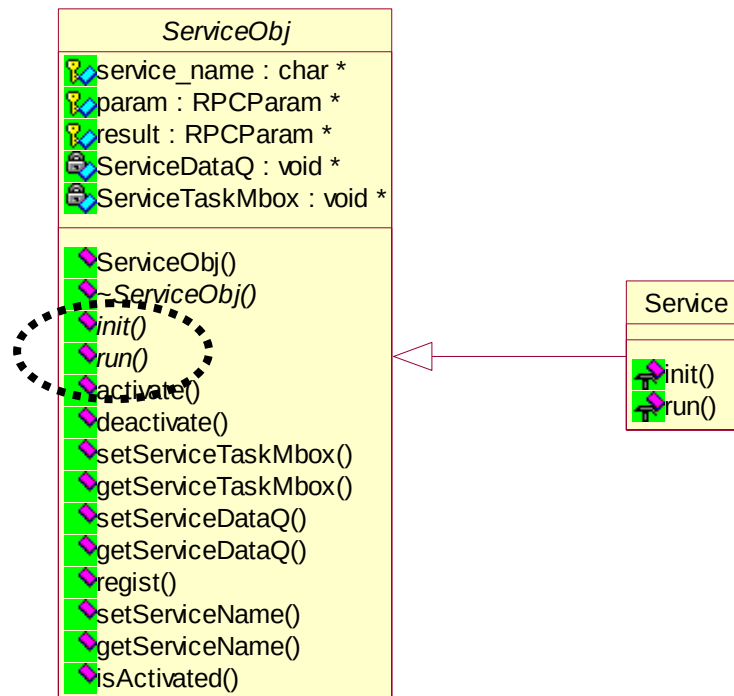
- โปรโตคอลซ็อกเก็ต (Protocol Socket)

เป็นคลาสที่จัดการการเชื่อมต่อของแต่ละเซสชัน โดยสร้างขึ้นมาเพื่อรองรับรูปแบบเซสชันที่หลากหลายตามแต่โปรโตคอล เพื่อให้ทำงานร่วมกับทาสก์และเซิร์ฟเวอร์แอดมอนได้อย่างถูกต้อง



รูปภาพ 4-5 ตัวอย่างการอิมพลีเมนต์ซ็อกเก็ต เพื่อทำงานร่วมกับเซิร์ฟเวอร์แอดมอนในแต่ละโปรโตคอล

4.2 นักพัฒนาบริการบนระบบฝังตัว



รูปภาพ 4-6 การสร้างบริการโดยการอิมพลิเมนต์เซอร์วิสออบเจกต์

ในการสร้างบริการหรือเซอร์โวลด์ เข้ามาใช้ในระบบนั้น นักพัฒนาจะต้องทำการอิมพลิเมนต์คลาสของเซอร์วิสออบเจกต์ ที่มีอยู่ในระบบ เพื่อใช้อินเตอร์เฟส (Interface) ที่จะเข้าไปทำงานร่วมกับระบบได้อย่างถูกต้อง โดยส่วนที่ต้องอิมพลิเมนต์เพิ่มนั้นมีดังนี้

- ฟังก์ชัน `init()`

เป็นฟังก์ชันที่ทำหน้าที่ในการแปลงข้อมูลจากพารามิเตอร์ที่ส่งเข้ามา ในรูปของสตริง (String) ไปเป็นข้อมูลในชนิดที่ระบบต้องการใช้ในการประมวลผล เพื่อเตรียมความพร้อมก่อนที่จะทำการประมวลผลต่อไป

- ฟังก์ชัน `run()`

เป็นฟังก์ชันหลัก ที่เป็นส่วนประมวลผลของบริการหรือเซอร์วิสนั้นๆ ในลักษณะของการทำงานแบบฟังก์ชันทั่วๆ ไป ที่ง่ายต่อการสร้างและพัฒนา โดยไม่ต้องสนใจส่วนอื่นๆ ในระบบ เพียงแต่เมื่อทำการประมวลผลฟังก์ชันเสร็จแล้ว ก็ให้แปลงข้อมูลจากชนิดที่เป็นอยู่ ไปเป็นสตริง แล้วนำไปเก็บไว้ที่อาร์พีซีรีซัลต์ เพื่อรอการนำไปจัดส่งผลลัพธ์กลับไปยังผู้ร้องขอตามโปรโตคอลต่างๆ ต่อไป

```

#ifndef CALCULATOR
#define CALCULATOR

#include "servobj.h"

class Calculator : public ServiceObj{ //extend and implement ServiceObject
public:

    Calculator(const char* serv_name):ServiceObj(serv_name){};
    ~Calculator(){};

    void init(RPCParam *p_param,RPCParam *p_result){
        //converse type from string to expecting type
    }

    void run() {
        // service process code

        // converse type to string

        // save in RPC Result
    }

};

#endif

```

รูปภาพ 4-7 ตัวอย่างการโค้ดในการสร้างเซอร์วิสในระบบ

หลังจากนักพัฒนาได้ทำการพัฒนาเซอร์วิสโค้ดเสร็จแล้ว ก่อนใช้งานนั้น ต้องมีการบันทึกรายละเอียดต่างๆที่เกี่ยวข้องกับเซอร์วิสที่ได้สร้างขึ้นมานั้น ได้แก่

- ชื่อเรียกของเซอร์วิส
- ชื่อเต็มของเซอร์วิส
- ข้อความอธิบายเซอร์วิสแบบย่อ
- สถานะในการเปิดใช้งานเมื่อระบบเริ่มต้น

ข้อมูลดังกล่าว จะถูกเก็บไว้ในไฟล์เซอร์วิสจิสทรี (Service Registry) ในรูปแบบของภาษาเอ็กซ์เอ็มแอล(XML Format) ไว้สำหรับตั้งค่าต่างๆให้กับเซอร์วิสที่ลงทะเบียนเพื่อเป็นข้อมูลในการเรียกใช้เซอร์วิสต่อไป นอกเหนือจากนี้ นักพัฒนาสามารถเพิ่มฟิลด์ข้อมูลอื่นๆ ที่เป็นลักษณะเฉพาะหรือมีความจำเป็นต้องใช้ในการทำงานของเซอร์วิสได้อีกด้วย

```

<Service type="registry">
  <cal type="key">
    <Name type="string">Calculator</Name>
    <Description type="string">Sample service as calculator</Description>
    <Status type="string">on</Status>
  </cal>
  <address type="key">
    <Name type="string">Address Lists</Name>
    <Description type="string">Sample service as address lists</Description>
    <Status type="string">off</Status>
  </address>
</Service>

```

รูปภาพ 4-8 ตัวอย่างเซอร์วิสรีจิสทรีไฟล์ที่เก็บในรูปแบบเอกสารเอ็กซ์เอ็มแอล

4.3 ผู้ที่เรียกใช้บริการของระบบ

ในลักษณะการใช้งานนี้ ผู้ที่เรียกใช้บริการเซอร์วิสจากระบบฝั่งตัวจะต้องทำการร้องขอรายละเอียดของเซอร์วิสก่อนเพื่อให้ทราบถึงข้อมูลเกี่ยวกับอินเตอร์เฟซในการติดต่อกับเซอร์วิส จากนั้นสามารถที่จะทำการสร้างแอปพลิเคชันจากไลบรารีต่าง ๆ ขึ้นมาได้ตามสภาพแวดล้อมการใช้งานของผู้ใช้ได้ เช่น อาปาเชโซบ คอทเน็ตเฟรมเวิร์ก เป็นต้น โดยผู้ใช้งานสามารถนำข้อมูลที่ได้รับกลับมาสร้างเป็นรูปแบบต่างๆ ได้ตามต้องการ อีกทั้งยังสามารถที่จะสร้างอินเตอร์เฟซได้เอง

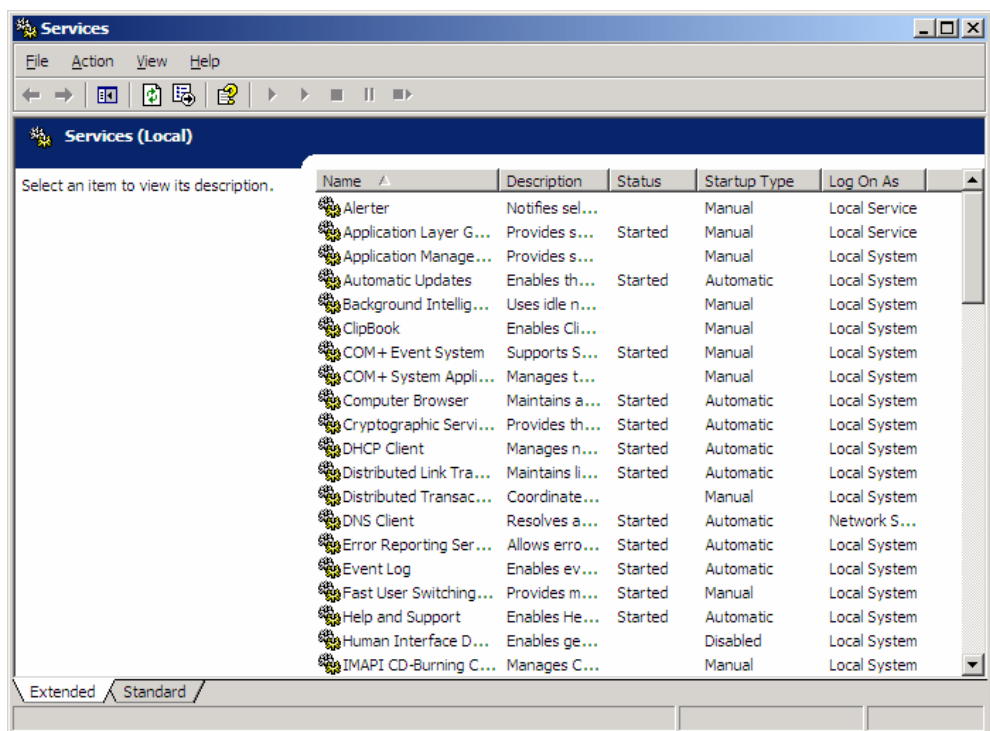
4.4 ผู้ดูแลระบบการให้บริการของระบบ

ภายในระบบ มีส่วนที่ทำหน้าที่ในการจัดการและควบคุมเซอร์วิสในระบบ โดยผู้ดูแลระบบสามารถพัฒนาแอปพลิเคชันที่ใช้ติดต่อกับเซอร์วิสเมนเจอร์ ในลักษณะของการเรียกใช้เซอร์วิสแบบหนึ่ง ซึ่งแอปพลิเคชันนั้น สามารถดัดแปลงรูปแบบได้หลากหลาย อาทิเช่น การเรียกใช้เซอร์วิสเมนเจอร์ผ่านเทลเน็ต (Telnet) ซึ่งสามารถให้แสดงผลในลักษณะแบบของโปรแกรมท๊อป (Top) ที่มีอยู่ในระบบปฏิบัติการลินุกซ์ (Linux) หรือยูนิกซ์ (Unix) หรืออาจเป็นแอปพลิเคชันที่พัฒนามบนแพลตฟอร์มของระบบปฏิบัติการในตระกูลวินโดวส์ (Windows OS Platform) ที่มีจุดเด่นในด้านของการแสดงผลแบบกราฟิกยูสเซอร์อินเตอร์เฟซ (Graphic User Interface)

11:53am up 6 days, 21:51, 1 user, load average: 0.01, 0.01, 0.00
 84 processes: 82 sleeping, 2 running, 0 zombie, 0 stopped
 CPU states: 0.0% user, 0.1% system, 0.0% nice, 99.7% idle
 Mem: 1030292K av, 994684K used, 35608K free, 202952K buff
 Swap: 2612408K av, 34004K used, 2578404K free, 604852K cached

PID	USER	PRI	NI	SIZE	RSS	SHARE	STAT	LIB	%CPU	%MEM	TIME	COMMAND
18193	owen	15	0	1460	1460	1260	R	0	0.3	0.1	0:00	top
1	root	15	0	476	448	424	S	0	0.0	0.0	0:13	init
2	root	0K	0	0	0	0	SW	0	0.0	0.0	0:00	migration/0
3	root	0K	0	0	0	0	SW	0	0.0	0.0	0:00	migration/1
4	root	15	0	0	0	0	SW	0	0.0	0.0	0:01	keventd
5	root	34	19	0	0	0	SWN	0	0.0	0.0	0:00	ksoftirqd_CPU
6	root	34	19	0	0	0	SWN	0	0.0	0.0	0:00	ksoftirqd_CPU
11	root	25	0	0	0	0	SW	0	0.0	0.0	0:00	bdfush
7	root	15	0	0	0	0	SW	0	0.0	0.0	0:47	kswapd
8	root	15	0	0	0	0	SW	0	0.0	0.0	0:08	kscand/DMA
9	root	16	0	0	0	0	SW	0	0.0	0.0	29:35	kscand/Normal
10	root	15	0	0	0	0	SW	0	0.0	0.0	9:49	kscand/HighMe
12	root	15	0	0	0	0	SW	0	0.0	0.0	0:15	kupdated
13	root	25	0	0	0	0	SW	0	0.0	0.0	0:00	mdrecoveryd
25	root	15	0	0	0	0	SW	0	0.0	0.0	0:53	raid5d
26	root	15	0	0	0	0	SW	0	0.0	0.0	0:03	raid5d
27	root	21	0	0	0	0	SW	0	0.0	0.0	0:00	raid1d
28	root	15	0	0	0	0	SW	0	0.0	0.0	0:45	kjournald

รูปภาพ 4-9 ตัวอย่างรูปแบบในการควบคุมเซอร์วิสในระบบของโปรแกรม TOP



รูปภาพ 4-10 ตัวอย่างรูปแบบในการควบคุมเซอร์วิสของเซอร์วิสเมนเจอร์ในวินโดวส์